Gathering System Information

Now that we are familiar with navigating our Windows host using nothing but the Command Prompt let us move on to a fundamental concept accessible to both `Systems Administrators` and `Penetration Testers`: `Gathering System Information`.

Gathering `system information`(aka `host enumeration`) may seem daunting at first; however, it is a crucial step in providing a good foundation for getting to know our environment. Learning the environment and getting a general feel for our surroundings is beneficial to both sides of the aisle, benefitting the `red team` and the `blue team`. Those seated on the `red team`(Penetration Testers, Red Team Operators, hackers, etc.) will find value in being able to scan their hosts and the environment to learn what vulnerable services and machines can be exploited. Whereas the `blue team`(System Administrators, SOC Analysts, etc.) can use the information to diagnose issues, secure hosts and services, and ensure integrity across the network. Regardless of which team we might find ourselves most interested in or currently involved in, this section aims to provide the following information:

- What information can we gather from the system(`host`)?
- Why do we need this information, and what is the importance of thorough enumeration?
- How do we get this information via Command Prompt, and what general methodology should we follow?

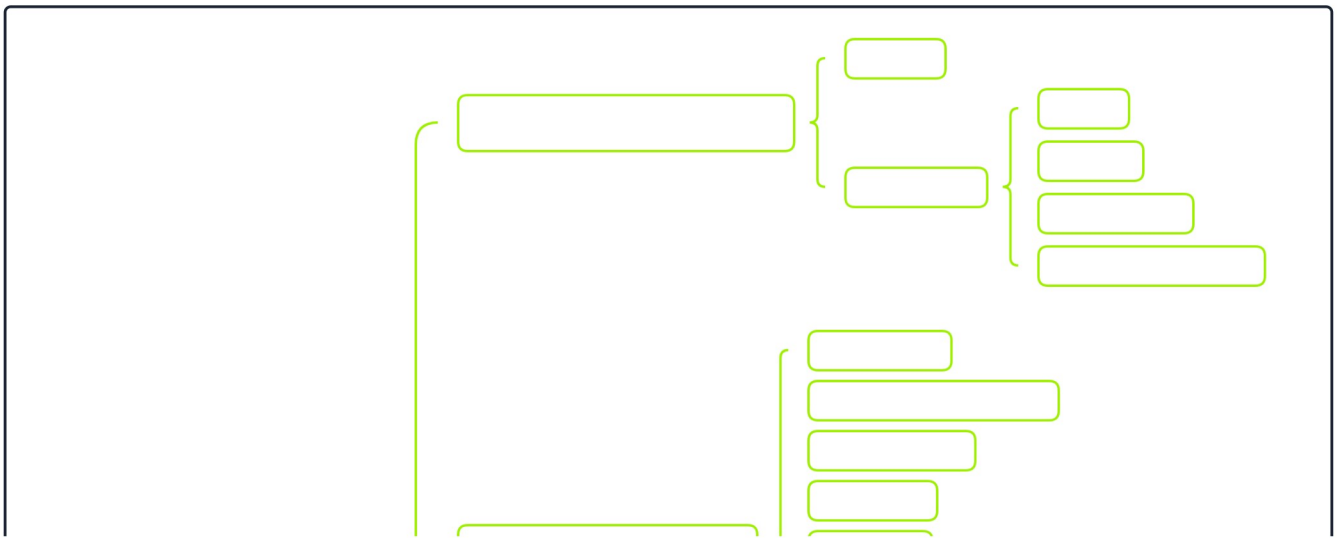
What Types of Information Can We Gather from the System?

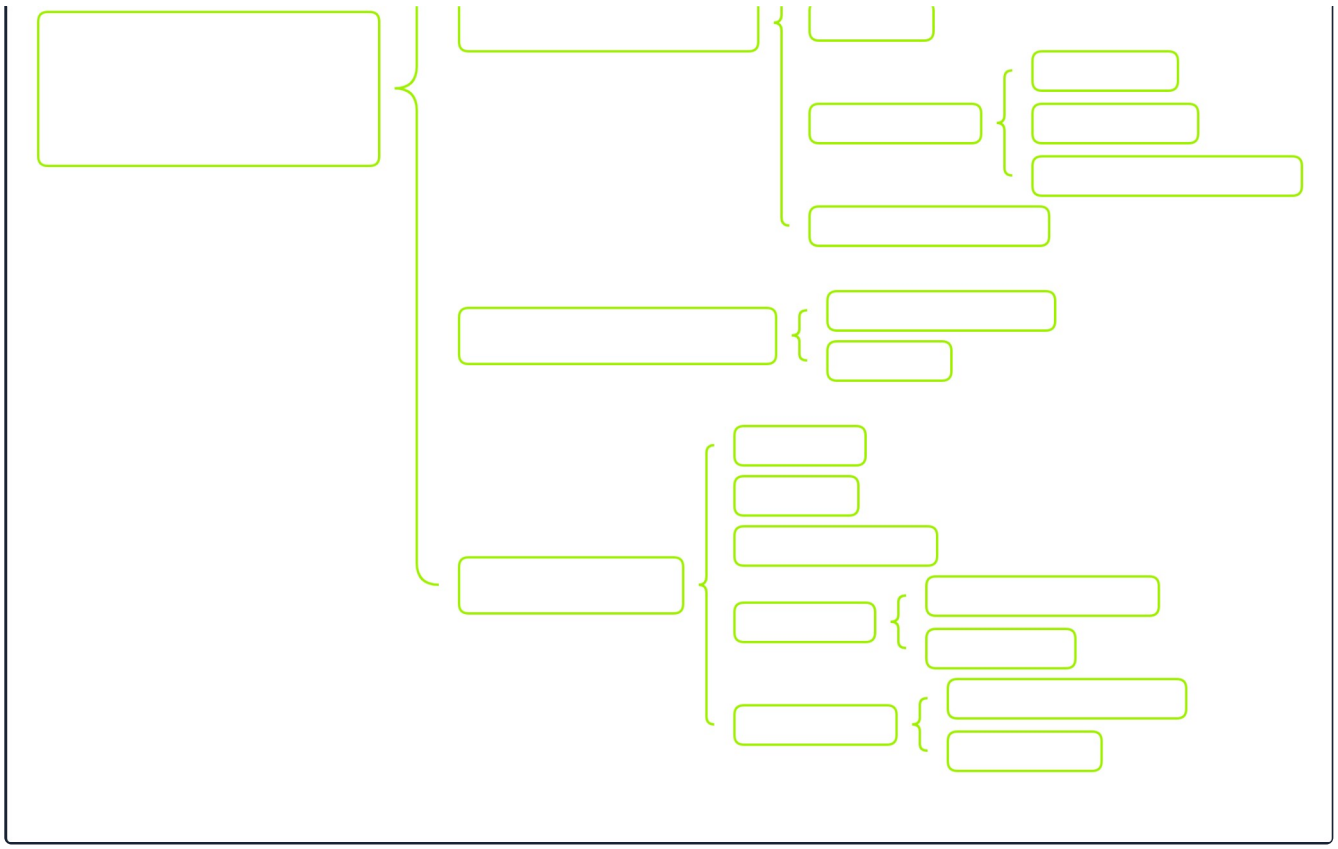
Once we have initial access to the system through some `command shell`, just knowing where to begin searching the system for information can be difficult. Manually `enumerating` the system with no path in mind on how we wish to proceed can lead to plenty of lost hours searching through troves of what seems to be important information with little to no results to show for all of that time spent. The goal of `host enumeration` is to provide an overall picture of the target host, its environment, and how it interacts with other systems across the network. Keeping this in mind, the first question that we might find ourselves coming to is:

`How do we know what to look for?`

To answer this question, we need to have a basic understanding of all the different types of information available to us on a system. Below is a chart that we can reference to give us a generalized outline of the main types of information we need to be aware of while performing host enumeration.

> *Note: This example is aimed toward the enumeration of the Windows operating system and may not be fully compatible with other system types. Also, note that this example is a partial list of all information found on a system.*





As we can see from the diagram above, the types of information that we would be looking for can be broken down into the following categories:

Type	Description
`General System Information`	Contains information about the overall target system. Target system information includes but is not limited to the `hostname` of the machine, OS-specific details (`name`, `version`, `configuration`, etc.), and `installed hotfixes/patches` for the system.
`Networking Information`	Contains networking and connection information for the target system and system(s) to which the target is connected over the network. Examples of networking information include but are not limited to the following: `host IP address`, `available network interfaces`, `accessible subnets`, `DNS server(s)`, `known hosts`, and `network resources`.
`Basic Domain Information`	Contains Active Directory information regarding the domain to which the target system is connected.
`User Information`	Contains information regarding local users and groups on the target system. This can typically be expanded to contain anything accessible to these accounts, such as `environment variables`, `currently running tasks`, `scheduled tasks`, and `known services`.

Although this is not an exhaustive list of every single piece of information on a system, this will provide us with the means to begin creating a solid methodology for enumeration. Peering back at the diagram with our newfound knowledge, we can see a pattern emerge as to what we should be looking for while performing enumeration on our target host. To keep ourselves on target during enumeration, we want to try and ask ourselves some of the following questions:

- `What system information can we pull from our target host?`
- `What other system(s) is our target host interacting with over the network?`
- `What user account(s) do we have access to, and what information is accessible from the account(s)?`

Think of these questions as a way to provide structure to help us develop a sense of situational awareness and a methodology for testing. Doing so gives us a clearer idea of what we are looking for and what information needs to be filtered out or prioritized during a real-life engagement.

Why Do We Need This Information?

Why Do We Need This Information?

In the previous section, we discussed what information can be gathered from a system during enumeration and what we should be aware of during our search. This section will provide more of the `why` behind gathering information in the first place and the importance of thorough enumeration of a target.

As stated beforehand, our `goal` with `host enumeration` here is to use the information gained from the target to provide us with a starting point and guide for how we wish to attack the system. To gain a better grasp on the concept behind the importance of proper host enumeration, let us follow along with the following example:

Example: Imagine you are tasked with working on an `assumed breach` engagement and have been provided with initial access through what is assumed to be an unprivileged user account. Your task is to get an overall lay of the land and see if you can `escalate your privileges` beyond the initial access of the compromised user account.

Following this example scenario, we can see that we are provided direct access to our initial host through an assumed unprivileged user account. However, our goal is to eventually escalate our privileges to an account with access to higher privileges or administrative permissions if we are lucky. To do this, we are going to need a thorough understanding of our environment, including the following:

- What user account do we have access to?
- What groups does our user belong to?
- What current working set of privileges does our user have access to?
- What resources can our user access over the network?
- What tasks and services are running under our user account?

Remember that this only partially encompasses all the questions we can ask ourselves to reach our intended goal but simply a tiny subset of possibilities. Without thinking things through and failing to follow any guided structure while performing `enumeration`, we will struggle to know if we have all the required information to reach our goal. It can be easy to write off a system as being completely patched and not vulnerable to any current CVEs or the latest `vulnerabilities`. However, if you only focus on that aspect, it is easy to miss out on the many human configuration errors that could exist in the environment. This very reason is why taking our time and gathering all of the information we can on a system or environment should be prioritized in terms of importance over simply exploiting a system haphazardly.

How Do We Get This Information?

Casting a Wide Net

CMD provides a one-stop shop for information via the `systeminfo` command. It is excellent for finding relevant information about the host, such as hostname, IP address(es), if it belongs to a domain, what hotfixes have been installed, and much more. This information is super valuable for a sysadmin when trying to diagnose issues.

For a hacker, this is a great way to quickly get the lay of the land when you first access a host while leaving a minimal footprint. Running one command is always better than running two or three just to get the same information. We are less likely to be detected this way. Having quick access to things such as the OS version, hotfixes installed, and OS build version can help us quickly determine from a quick Google or ExploitDB search, if an exploit exists that can be quickly leveraged to exploit this host further, elevate privileges, and more.

Systeminfo Output

Gathering System Information

```
C:\htb> systeminfo
```

```
C:\htb> systeminfo

Host Name:                 DESKTOP-htb
OS Name:                   Microsoft Windows 10 Pro
OS Version:                10.0.19042 N/A Build 19042
OS Manufacturer:           Microsoft Corporation
OS Configuration:          Standalone Workstation
OS Build Type:             Multiprocessor Free

<snipped>
```

However, knowing a single way to gather information is inefficient, especially if specific commands are monitored and tracked more closely than others. This is why we need more than one established way to gather our required information and stay under the detection radar when possible.

Examining the System

As shown previously, `systeminfo` contains a lot of information to sift through; however, if we need to retrieve some basic system information such as the `hostname` or `OS version`, we can use the `hostname` and `ver` utilities built into the command prompt.

The hostname utility follows its namesake and provides us with the hostname of the machine, whereas the ver command prints out the current operating system version number. Both commands, in tandem, will provide us with an alternative way to retrieve some basic system information we can use while further enumerating the target host.

Hostname Output

Gathering System Information

```
C:\htb> hostname

DESKTOP-htb
```

Ver Output

Gathering System Information

```
C:\htb> ver

Microsoft Windows [Version 10.0.19042.2006]
```

Scoping the Network

In addition to the host information provided above, let us quickly look at some basic network information for our target. A thorough understanding of how our target is connected and what devices it can access across the network is an invaluable tool in our arsenal as an attacker.

To gather this information quickly and in one simple-to-use command, Command Prompt offers the `ipconfig` utility. The ipconfig utility displays all current TCP/IP network configurations for the machine. Let us look at an example `ipconfig` configuration without providing additional parameters.

Ipconfig Without Parameters

Gathering System Information

```
C:\htb> ipconfig
```

```
Windows IP Configuration

<SNIP>

Ethernet adapter Ethernet:

   Connection-specific DNS Suffix  . : htb.local
   Link-local IPv6 Address . . . . . : fe80::2958:39a:df51:b60%23
   IPv4 Address. . . . . . . . . . . : 10.0.25.17
   Subnet Mask . . . . . . . . . . . : 255.255.255.0
   Default Gateway . . . . . . . . . : 10.0.25.1

Ethernet adapter Ethernet 2:

   Connection-specific DNS Suffix  . : internal.htb.local
   Link-local IPv6 Address . . . . . : fe80::bc3b:6f9f:68d4:3ec5%26
   IPv4 Address. . . . . . . . . . . : 172.16.50.15
   Subnet Mask . . . . . . . . . . . : 255.255.255.0
   Default Gateway . . . . . . . . . : 172.16.50.1

<SNIP>
```

As we can see from the example above, even without specifying parameters, we are greeted with some basic network information for the host machine, such as the `Domain Name`, `IPv4 Address`, `Subnet Mask`, and `Default Gateway`. All of these can provide insight into the network(s) that the target is a part of and connected to and the wider environment. If we need additional information or want to dig further into the specific settings applied to each adapter, we can use the following command: `ipconfig /all`. As implied by the flag provided, this command provides a fully comprehensive listing (full TCP/IP configuration) of every network adapter attached to the system and additional information, including the physical address of each adapter (`MAC Address`), DHCP settings, and DNS Servers.

`Ipconfig` is a highly versatile command for gathering information about the network connectivity of the target host; however, if we need to quickly see what hosts our target has come into contact with, look no further than the `arp` command.

The arp utility effectively displays the contents and entries contained within the Address Resolution Protocol (`ARP`) cache. We can also use this command to modify the table entries effectively. However, that in itself is beyond the scope of this module. To better understand what type of information the `ARP` cache contains, let us quickly look at the following example:

Utilizing ARP to Find Additional Hosts

Gathering System Information

```
C:\htb> arp /a

<SNIP>

Interface: 10.0.25.17 --- 0x17
  Internet Address      Physical Address      Type
  10.0.25.1             00-e0-67-15-cf-43     dynamic
  10.0.25.5             54-9f-35-1c-3a-e2     dynamic
  10.0.25.10            00-0c-29-62-09-81     dynamic
  10.0.25.255           ff-ff-ff-ff-ff-ff     static
  224.0.0.22            01-00-5e-00-00-16     static
  224.0.0.251           01-00-5e-00-00-fb     static
  224.0.0.252           01-00-5e-00-00-fc     static
  239.255.255.250       01-00-5e-7f-ff-fa     static
  255.255.255.255       ff-ff-ff-ff-ff-ff     static

Interface: 172.16.50.15 --- 0x1a
  Internet Address      Physical Address      Type
  172.16.50.1           15-c0-6b-58-70-ed     dynamic
  172.16.50.20          80-e5-53-3c-72-30     dynamic
  172.16.50.32          fb-90-01-5c-1f-88     dynamic
```

```
  172.16.50.65          7a-49-56-10-3b-76     dynamic
  172.16.50.255         ff-ff-ff-ff-ff-ff     static
  224.0.0.22            01-00-5e-00-00-16     static
  224.0.0.251           01-00-5e-00-00-fb     static
  224.0.0.252           01-00-5e-00-00-fc     static
  239.255.255.250       01-00-5e-7f-ff-fa     static\

<SNIP>
```

From this example, we can see all the hosts that have come into contact or might have had some prior communication with our target. We can use this information to begin mapping the network along each of the networking interfaces belonging to our target.

Understanding Our Current User

Now that we have some basic host information to get us started, we should further understand our current compromised user account. One of the best command line utilities to do so is `whoami`.

Whoami allows us to display the user, group, and privilege information for the user that is currently logged in. In this case, we should run it without any parameters first and see what kind of output we end up with.

Gathering System Information

```
C:\htb> whoami

ACADEMY-WIN11\htb-student
```

As we can see from the initial output above, running `whoami` without parameters provides us with the current domain and the user name of the logged-in account.

Note: If the current user is not a domain-joined account, the `NetBIOS` name will be provided instead. The current `hostname` will be used in most cases.

Checking Out Our Privileges

As previously mentioned, we can also use `whoami` to view our current user's security privileges on the system. By understanding what privileges are enabled for our current user, we can determine our capabilities on our target host. Let us try running `whoami /priv` from our compromised user account.

Gathering System Information

```
C:\htb> whoami /priv

PRIVILEGES INFORMATION
----------------------

Privilege Name                Description                          State
============================= ==================================== ========
SeShutdownPrivilege           Shut down the system                 Disabled
SeChangeNotifyPrivilege       Bypass traverse checking             Enabled
SeUndockPrivilege             Remove computer from docking station Disabled
SeIncreaseWorkingSetPrivilege Increase a process working set       Disabled
SeTimeZonePrivilege           Change the time zone                 Disabled
```

From the output above, we only seem to have access to a basic permission set, and most of our options are disabled. This falls within the limitations of a standard user account provisioned on the domain. However, if there were any misconfigurations in these settings or the user was provided any additional privileges, we could potentially use this to our advantage in trying to escalate the privileges of

our current user.

Investigating Groups

On top of having a thorough understanding of our current user's privileges, we should also take some time to see what groups our account is a member of. This can provide insight into other groups our current user is a part of, including any default groups (built-ins) and, more importantly, any custom groups to which our user was explicitly granted access. To view the groups our current user is a part of, we can issue the following command: `whoami /groups`.

Gathering System Information

```
C:\htb> whoami /groups

GROUP INFORMATION
-----------------

Group Name                             Type             SID          Attributes
====================================== ================ ============ ==========================================
Everyone                               Well-known group S-1-1-0      Mandatory group, Enabled by default, Enab
BUILTIN\Users                          Alias            S-1-5-32-545 Mandatory group, Enabled by default, Enab
BUILTIN\Performance Log Users          Alias            S-1-5-32-559 Mandatory group, Enabled by default, Enab
NT AUTHORITY\INTERACTIVE               Well-known group S-1-5-4      Mandatory group, Enabled by default, Enab
CONSOLE LOGON                          Well-known group S-1-2-1      Mandatory group, Enabled by default, Enab
NT AUTHORITY\Authenticated Users       Well-known group S-1-5-11     Mandatory group, Enabled by default, Enab
NT AUTHORITY\This Organization         Well-known group S-1-5-15     Mandatory group, Enabled by default, Enab
NT AUTHORITY\Local account             Well-known group S-1-5-113    Mandatory group, Enabled by default, Enab
LOCAL                                  Well-known group S-1-2-0      Mandatory group, Enabled by default, Enab
NT AUTHORITY\NTLM Authentication       Well-known group S-1-5-64-10  Mandatory group, Enabled by default, Enab
Mandatory Label\Medium Mandatory Level Label            S-1-16-8192
```

Our user is not a member of any other groups besides the built-ins added to our account upon creation. However, it is essential to note that in some cases, users can be provided additional access, privileges, and permissions based on the groups to which they belong.

Note: The commands shown above contain only certain sections of the output provided from `whoami /all`. Depending on the situation and the information that's needed, we can use the individual commands or gather all of the information at once through the use of the `/all` parameter.

Investigating Other Users/Groups

After investigating our current compromised user account, we need to branch out a bit and see if we can get access to other accounts. In most environments, machines on a network are domain-joined. Due to the nature of domain-joined networks, anyone can log in to any physical host on the network without requiring a local account on the machine. We can use this to our advantage by scoping out what users have accessed our current host to see if we could access other accounts. This is very beneficial as a method of maintaining persistence across the network. To do this, we can utilize specific functionality of the `net` command.

Net User

Net User allows us to display a list of all users on a host, information about a specific user, and to create or delete users.

Gathering System Information

```
C:\htb> net user

User accounts for \\ACADEMY-WIN11

-------------------------------------------------------------------------------
Administrator            DefaultAccount           Guest
htb-student              WDAGUtilityAccount
The command completed successfully.
```

```
The command completed successfully.
```

From the provided output, only a few user accounts have been created for this machine. However, if we were on a more populated network, we might come across more accounts to attempt to compromise.

Net Group / Localgroup

In addition to user accounts, we should also take a quick look into what groups exist across the network. In the previous section, we discussed very heavily into groups that our user is a member of; however, we also have the capability from our current host to view all groups that exist on our host and from the domain. We can achieve this by utilizing the `net group` and `net localgroup` commands.

Net Group will display any groups that exist on the host from which we issued the command, create and delete groups, and add or remove users from groups. It will also display domain group information if the host is joined to the domain. Keep in mind, `net group` must be run against a domain server such as the DC, while `net localgroup` can be run against any host to show us the groups it contains.

Gathering System Information

```
C:\htb> net group
net group
This command can be used only on a Windows Domain Controller.

More help is available by typing NET HELPMSG 3515.


C:\htb>net localgroup

Aliases for \\ACADEMY-WIN11

-------------------------------------------------------------------------------
*__vmware__
*Access Control Assistance Operators
*Administrators
*Backup Operators
*Cryptographic Operators
*Device Owners
*Distributed COM Users
*Event Log Readers
*Guests
*Hyper-V Administrators
*IIS_IUSRS
*Network Configuration Operators
*Performance Log Users
*Performance Monitor Users
*Power Users
*Remote Desktop Users
*Remote Management Users
*Replicator
*System Managed Accounts Group
*Users
The command completed successfully.
```

Exploring Resources on the Network

Previously, we honed in on what our current user has access to in terms of the host locally. However, in a domain environment, users are typically required to store any work-related material on a share located on the network versus storing files locally on their machine. These shares are usually found on a server away from the physical access of a run-of-the-mill employee. Typically, standard users will have the necessary permissions to read, write, and execute files from a share, provided they have valid credentials. We can, of course, abuse this as an additional persistence method, but how do we locate these shares in the first place?

Net Share

One way of doing so involves using the `net share` command. Net Share allows us to display info about shared resources on the host and to create new shared resources as well.

Gathering System Information

```
C:\htb> net share

Share name   Resource                        Remark

-------------------------------------------------------------------------------
C$           C:\                             Default share
IPC$                                         Remote IPC
ADMIN$       C:\Windows                      Remote Admin
Records      D:\Important-Files              Mounted share for records storage
The command completed successfully.
```

As we can see from the example above, we have a list of shares that our current compromised user has access to. By reading the remarks, we can take an educated guess that `Records` is a manually mounted share that could contain some potentially interesting information for us to enumerate. Ideally, if we were to find an open share like this while on an engagement, we would need to keep track of the following:

- Do we have the proper permissions to access this share?
- Can we read, write, and execute files on the share?
- Is there any valuable data on the share?

In addition to providing information, `shares` are great for hosting anything we need and laterally moving across hosts as a pentester. If we are not too worried about being sneaky, we can drop a payload or other data onto a share to enable movement around other hosts on the network. Although outside of the scope of this module, abusing shares in this manner is an excellent persistence method and can potentially be used to escalate privileges.

Net View

If we are not explicitly looking for shares and wish to search the environment broadly, we have an alternate command that can be extremely useful, also known as `net view`.

Net View will display to us any shared resources the host you are issuing the command against knows of. This includes domain resources, shares, printers, and more.

Gathering System Information

```
C:\htb> net view
```

Piecing Things Together

Using all the information and examples provided above, we have extracted a significant amount of information from our host and its' surroundings. From here, depending on our access, we can elevate our privileges or continue to move towards our goal. System-level access on every host is unnecessary for a pentester (unless the assessment calls for it), so let us avoid getting stuck trying to get it on every occasion.

This is just a quick look at how `CMD` can be used to gain access and continue an assessment with limited resources. Keep in mind that this route is quite noisy, and we will be noticed eventually by even a semi-competent blue team. As it stands, we are writing tons of logs, leaving traces across multiple hosts, and have little to no insight into what their `EDR` and `NIDS` was able to see.

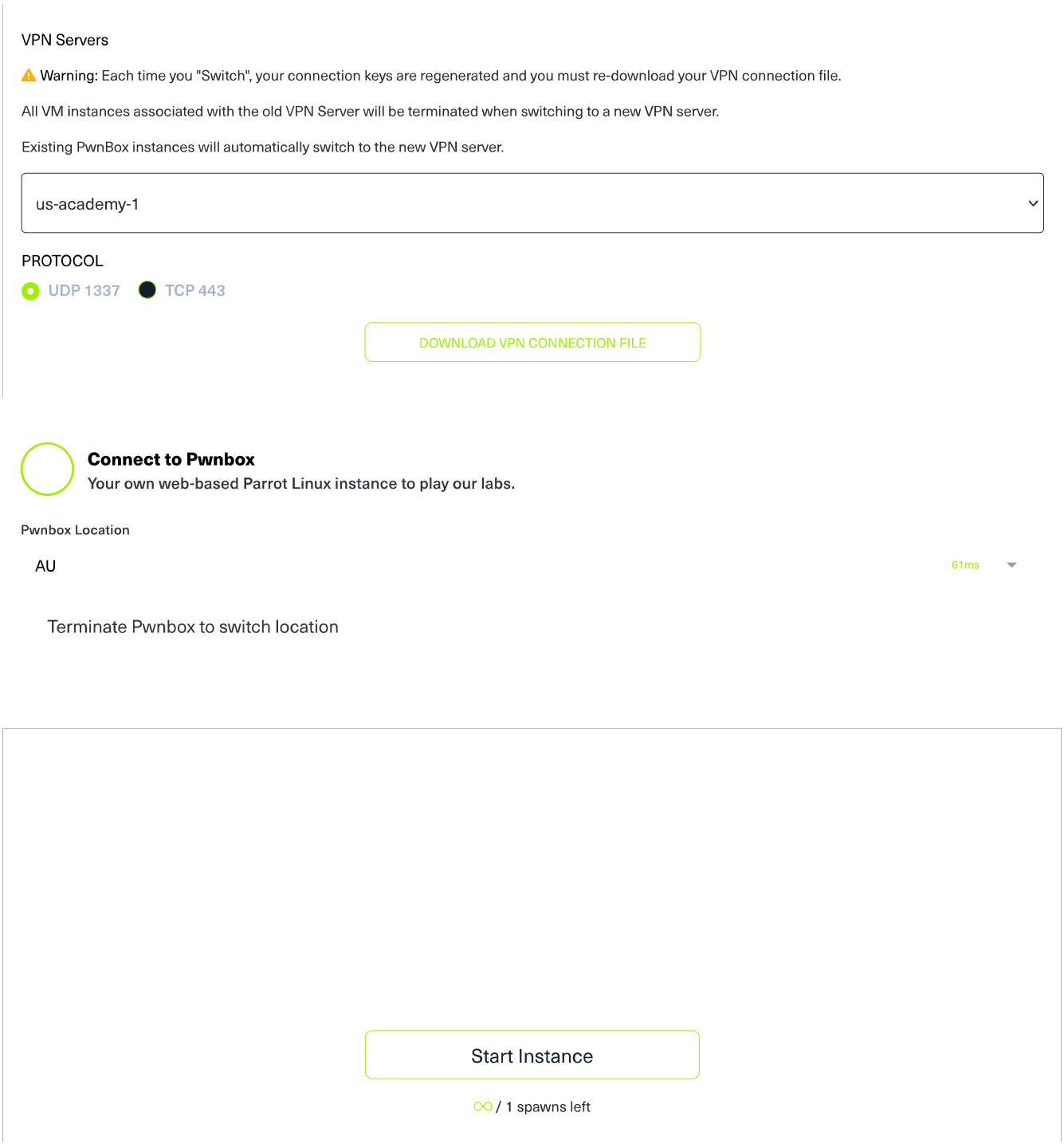
Note: In a standard environment, cmd-prompt usage is not a common thing for a regular user. Administrators sometimes have a

reason to use it but will be actively suspicious of any average user executing cmd.exe. With that in mind, using `net *` commands within an environment is not a normal thing either, and can be one way to alert on potential infiltration of a networked host easily. With proper monitoring and logging enabled, we should spot these actions quickly and use them to triage an incident before it gets too far out of hand.

Final Thoughts and Considerations

Albeit this has been an incredibly long section, we should have a general sense of the overall scope of information that can be found on a system, why we need it, and how we can gather this information quickly and efficiently. As a pentester, having this mindset allows us to further our methodology and gain a thorough understanding of what exactly we are looking for while enumerating a system or its wider environment. Having a solid methodology for enumeration is a highly invaluable skill for us to pick up early on.

As we move on to further sections in this module, our mindset and methodology will remain the same as we will simply be building upon the foundation being laid. In the next section, we will dive further into finding specific files and directories on our system.



Waiting to start...

Questions

Answer the question(s) below to complete this Section and earn cubes!

Target: Click here to spawn the target system!

Cheat Sheet

Download VPN Connection File

+ 0 What command will output verbose system information such as OS configuration, security info, hardware info, and more?

Submit your answer here...

Submit Hint

SSH to with user "htb-student" and password "HTB_@cademy_stdnt!"

+ 0 Access the target host and run the 'hostname' command. What is the hostname?

Submit your answer here...

Submit Hint

Previous Next

Cheat Sheet

Resources

Go to Questions

Table of Contents

- Environment Variables
- Managing Services
- Working With Scheduled Tasks

PowerShell

- CMD Vs. PowerShell
- All About Cmdlets and Modules
- User and Group Management
- Working with Files and Directories
- Finding & Filtering Content
- Working with Services
- Working with the Registry
- Working with the Windows Event Log
- Networking Management from The CLI
- Interacting with The Web
- PowerShell Scripting and Automation

Finish Strong

- Skills Assessment

Beyond This Module

My Workstation

OFFLINE

Start Instance

∞ / 1 spawns left